

ReelWalk cheat sheet

Five pages. Say the bold English line first, then the Japanese. If you blank, find the green picture: it brings the rest back.

What it is どんなアプリ？

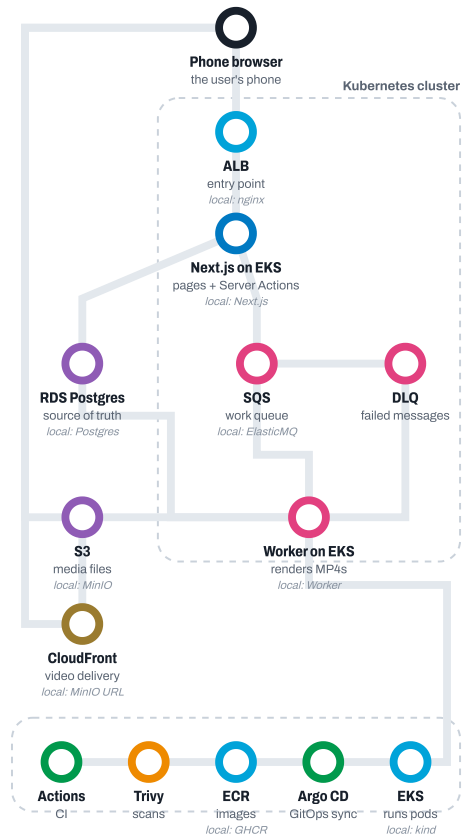
Phone in, Reel out.

- 1. ReelWalk makes Instagram Reels from property photos, on a phone.**
ReelWalkは、物件の写真から、スマホでリール動画を作るアプリです。
- 2. Its special feature is a floor-plan marker that moves with the video.**
特徴は、動画と一緒に動く、間取り図のマーカーです。
- 3. Restaurants can make Reels too, from their menu and their own clips.**
レストランも、メニューと自分の動画からリールを作れます。
- 4. A finished video can be shared with a link that works anywhere.**
できた動画は、共有リンクで、どこでも見られます。
- 5. It's live today at reelwalking.com.**
今は、reelwalking.comで本当に動いています。

How it works どう動く？

Web is the waiter, the queue is the order ticket, the worker is the kitchen.

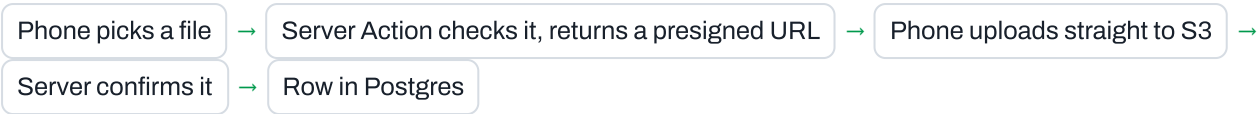
- 6. The web app is the waiter, the queue is the order ticket, the worker is the kitchen.**
Webはウェイター、キューは注文票、ワーカーはキッチンです。
- 7. The web app never makes videos. It only takes the order.**
Webアプリは、動画を作りません。注文を受けるだけです。
- 8. The database is the only source of truth. The queue only carries an ID.**
正しい情報はデータベースだけ。キューにはIDしか入れません。
- 9. Videos upload straight from the phone to S3.**
動画は、スマホからS3へ直接アップロードします。
- 10. While editing, the phone plays a light 720p copy; the export uses the original.**
編集中は軽い720pのコピーを再生して、書き出しは元の動画を使います。
- 11. Preview and export use the same component, so what you see is what you get.**
プレビューと書き出しは同じ部品なので、見たままの動画になります。



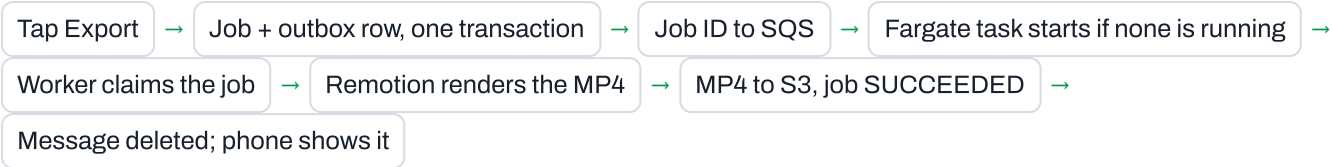
Web takes the order, SQS holds the ticket, the worker cooks, S3 keeps the dish, Postgres keeps the books.

The four flows in one line each

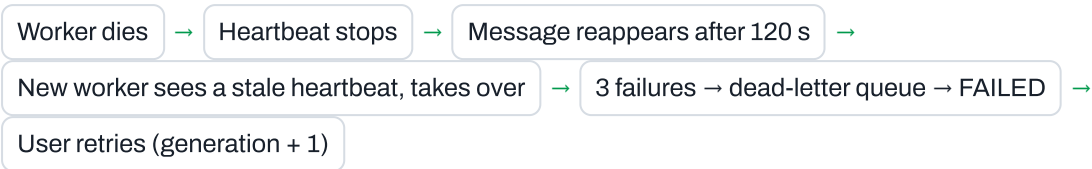
Upload



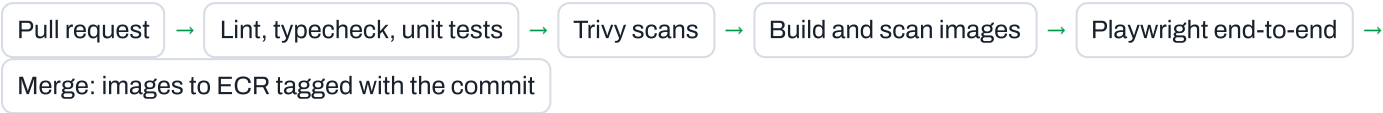
Export



Crash



Deploy



Numbers to know

Visibility timeout / heartbeat / stale after	120 s / every 20 s / 60 s
Attempts before the dead-letter queue	3 (backoff 15 s, then 30 s)
EKS for one day → today	~\$250/month → ~\$10–15/month
Idle worker exits after / cold start	4 minutes / 1–2 minutes
Load test	90 of 90 videos correct, with a worker killed
Editing preview / export source	720p copy / the original
Instagram Reel length / caption	3 s – 3 min / 2,200 characters
Upload limit / login session	2 GB / 30 days

If you only remember one thing: the web app never renders. It saves the job and its message in one transaction, sends only the job ID, and a worker that can crash safely does the heavy work. Everything else follows from that.

When you need a second

- 少し考えてもよろしいでしょうか。 *May I think for a moment?*
- もう一度おっしゃっていただけますか。 *Could you say that once more?*
- 結論から言うと、 *To give the conclusion first,*
- つまり、～ということですね。 *So you mean ..., right?*
- 正直に言うと、そこはまだ試していません。 *To be honest, I haven't tried that yet.*
- 例えば、ReelWalkでは、 *For example, in ReelWalk,*

1. I designed it assuming a message can arrive twice.

メッセージは二回届くかもしれない、と考えて作りました。

2. If it runs twice, there's still one result. That's idempotency.

二回動いても、結果は一つです。これが冪等性^{べきとうせい}です。

3. If a worker dies, another one takes over within about two minutes.

ワーカーが止まっても、二分ぐらいで別のワーカーが引き継ぎます。

4. After three failures the job is marked failed, and the user can retry.

三回失敗したら、ジョブは失敗になって、ユーザーがやり直せます。

5. The job and its message are saved in the same transaction (an outbox).

ジョブとメッセージは、同じトランザクションで保存します。

6. In a load test, I made 90 videos and killed a worker midway. All 90 succeeded.

テストで90本作って、途中でワーカーを止めても、全部成功しました。

AWS and money AWSとお金

Built it big, measured it, rebuilt it small.

7. First I ran it on EKS for a day, then I looked at the bill.

最初はEKSで一日動かして、それから請求額^{せいきゅうがく}を見ました。

8. \$250 a month for something mostly idle. That didn't fit.

ほとんど使わないのに、月250ドル。それは合いませんでした。

9. Now it's App Runner plus Fargate only when needed: about \$10 to \$15 a month.

今はApp Runnerと、必要な時だけ動くFargateで、月10ドルから15ドルくらいです。

10. When nobody is exporting, no render server is running at all.

書き出しがない時は、サーバーは一台も動いていません。

11. The trade-off: the first export after a quiet spell takes a minute or two to start.

その代わり、最初の書き出しは、始まるまで一、二分かかります。

12. Kubernetes isn't gone: it still runs locally on kind, with Argo CD.

Kubernetesは、今もローカルのkindで、Argo CDと一緒に動いています。

If they ask... say

1. Why is the render worker separate? なぜワーカーを分けたんですか。

Rendering is heavy and slow, so it must never block the web app. They scale and fail separately.

動画作りは重くて時間がかかるので、Webと分けました。別々に増やせて、片方が落ちて大丈夫です。

2. Why SQS? なぜSQSですか。

Retries, a lease and a dead-letter queue come built in. The cost is duplicate messages, so the worker is idempotent.

リトライとデッドレターキューが最初からあるからです。その代わり重複ちようぶくがあるので、二回動いても大丈夫に作りました。

3. What if a worker crashes mid-render? 途中でワーカーが落ちたら、どうなりますか。

Its heartbeat stops, the message comes back after two minutes, and another worker takes over. After three tries the job fails and the user can retry.

ハートビートが止まって、二分後にメッセージが戻り、別のワーカーが続けます。三回だめなら失敗にして、ユーザーがやり直せます。

4. How do you avoid two results for one job? 結果が二つにならないように、どうしていますか。

Status changes are conditional updates, the file is named after the job, and there's one output row per job.

状態の更新に条件を付けて、ファイル名をジョブIDにしています。だから、結果は一つです。

5. Why PostgreSQL and Prisma? なぜPostgreSQLとPrismaですか。

The data is relational, and Prisma gives the web app and the worker one typed schema.

データ同士に関係があるからです。Prismaで、Webとワーカーが同じ型を使えます。

6. Why Server Components and Server Actions? なぜServer ComponentsとServer Actionsですか。

Pages read the database directly, and every change is a typed function checked with zod. No separate API to maintain.

画面からそのままデータを読めて、更新は型のある関数です。別のAPIを作らなくていいからです。

7. Why did you stop using EKS? なぜEKSをやめたんですか。

It worked, but it cost about \$250 a month for an app that's mostly idle. App Runner plus on-demand Fargate costs \$10 to \$15.

ちゃんと動きましたが、ほとんど使わないのに月250ドルでした。今は月10ドルから15ドルくらいです。

8. How do you deploy? どうやってデプロイしていますか。

CI tests and scans every change, then pushes images tagged with the commit to ECR. Production runs those images.

CIでテストとスキャンをして、コミットIDのタグでECRにイメージを送ります。本番はそのイメージを使います。

9. How do you handle vulnerabilities? 脆弱性ぜいじゃくせいには、どう対応していますか。

Trivy scans dependencies, config and images in CI. Each finding is fixed, or accepted with a written reason.

CIでTrivyがスキャンします。直すか、受け入れる理由を書いて残します。

10. Did you use AI to build it? AIを使って作りましたか。

Yes, as a tool. Every design decision is written down, and I can explain each one.

はい、道具として使いました。設計の判断は全部書いてあって、一つずつ説明できます。

11. What would you improve next? 次に何を良くしたいですか。

A faster first export, scaling workers by queue length, and Instagram posting out of development mode.

最初の書き出しを速くすること、キューの長さでワーカーを増やすこと、インスタ投稿を本番にすることです。

- 1. Every change is tested and scanned before it ships.**
変更は全部、テストとスキャンをしてから出します。
- 2. All the AWS resources are managed in Terraform.**
AWSのリソースは、全部Terraformで管理しています。
- 3. There are no access keys in production, only roles.**
本番にアクセスキーはありません。ロールだけです。
- 4. Each vulnerability found is either fixed, or accepted with a written reason.**
見つけた脆弱性は、直すか、理由を書いて受け入れます。
- 5. Rules are tested with Vitest, and user flows with Playwright, right up to a real export.**
ルールはVitestで、画面の流れはPlaywrightで、本物の書き出しまで確かめます。

Honest and forward-looking 正直に、前向きに

Say what's done, say what's next.

- 6. I keep what's done and what's not done clearly separate.**
できたことと、まだのことを、はっきり分けて話します。
- 7. Direct posting to Instagram is built, but it's still in Meta's development mode.**
インスタへの直接投稿は作りましたが、まだMetaの開発モードです。
- 8. I used AI tools too, but every design decision is written down.**
AIツールも使いましたが、設計の判断は全部、文書に残しています。
- 9. My biggest lesson: measure first, then choose.**
一番の学びは、測ってから選ぶことです。
- 10. Next, I want to make that first export start faster.**
次は、最初の書き出しを、もっと速くしたいです。

Decisions in one line

- **SQS for the render queue.** I used SQS because renders are long and can crash, and SQS gives me a lease, retries and a dead-letter queue for free; the cost is at-least-once delivery, so the worker is idempotent.
- **Separate, idempotent render workers.** The worker is separate because rendering scales and fails differently from the web app, and it's idempotent because the queue can deliver a job twice.
- **One composition for preview and export; edits as a document.** Preview and export use the same React component, so the video always matches the screen, and the edit is a JSON document changed by pure functions, which makes undo, autosave and tests simple.
- **Transactional outbox for render queue messages.** The job and its queue message are written in one database transaction and the message is sent afterwards, so a job can never be saved without its message; duplicates are fine because the worker is idempotent.
- **Production for under \$30 a month.** I moved the web app to App Runner and made the render worker a Fargate task the web app starts on demand, cutting the bill from about \$250 to about \$15 a month with the same images and queue.